Health Monitoring Node

Treat the **Health Monitoring node** as a producer of **signal-shaped records** that can be written directly into the `signals` table. For the health/device signals in your catalogue, the important fields already align with the signal model: `signal_type`, source=`system`, typed value column, unit, granularity, and metadata. Relevant examples in your catalogue include `device_heartbeat`, `device_connectivity_status`, `battery_level`, `power_supply_status`, `camera_obstruction_status`, `frame_brightness_contrast_quality`, `sensor_health_status`, `sensor_communication_status`, `communication_timeout_threshold`, and `battery_low_threshold`.

1. Output record format from Health Monitoring node

Every emitted record should map 1:1 to a row in `signals`.

Field	Meaning
`device_id`	Device producing the signal
`dog_id`	Usually `null` for health/system signals
`signal_type`	Canonical signal key
`signal_value_numeric`	Used for numeric signals
`signal_value_boolean`	Used for boolean signals
`signal_value_text`	Used for enum/text signals
`unit`	`percent`, `none`, `score`, `seconds`, etc.
`observed_at`	When the state/value was observed on device
`ingested_at`	When the node writes/publishes it
`source`	`system`
`confidence`	Usually `null` for deterministic health checks
`metadata`	Context for debugging and downstream logic

2. Signal-by-signal emission rules

Signal Type	Value Column	Emit When	Push Frequency
`device_heartbeat`	`signal_value_boolean` or numeric sequence	Every heartbeat cycle	Periodic
`device_connectivity_status`	`signal_value_text`	On state change	Change-based
`battery_level`	`signal_value_numeric`	On significant change + periodic snapshot	Hybrid
`power_supply_status`	`signal_value_text`	On state change	Change-based
`camera_obstruction_status`	`signal_value_boolean`	On state change	Change-based
`frame_brightness_contrast_quality`	`signal_value_numeric`	Periodic sampled update	Periodic
`sensor_health_status`	`signal_value_text`	On state change	Change-based
`sensor_communication_status`	`signal_value_text`	On state change	Change-based

Signal Type	Value Column	Emit When	Push Frequency
`communication_timeout_threshold`	`signal_value_numeric`	On config load/change	Config-change only
`battery_low_threshold`	`signal_value_numeric`	On config load/change	Config-change only

3. Recommended generation policy

Use **three emission modes** only.

A. Periodic

For signals that are naturally sampled:

- `device_heartbeat`
- `battery_level`
- `frame_brightness_contrast_quality`

B. Change-based

For state/status signals:

- `device_connectivity_status`
- `power_supply_status`
- `camera_obstruction_status`
- `sensor_health_status`
- `sensor_communication_status`

C. Config-change only

For thresholds/config values:

- `communication_timeout_threshold`
- `battery_low_threshold`

4. Recommended default frequencies for MVP

Signal Type	Recommended Frequency
`device_heartbeat`	every 5–15 sec
`battery_level`	every 60 sec, plus on ≥1–2% change
`frame_brightness_contrast_quality`	every 30–60 sec or when camera session active
`device_connectivity_status`	only on change
`power_supply_status`	only on change
`camera_obstruction_status`	only on change
`sensor_health_status`	only on change
`sensor_communication_status`	only on change
`communication_timeout_threshold`	on boot/config change

Signal Type	Recommended Frequency
`battery_low_threshold`	on boot/config change

5. Write policy to storage

Do **not** write every internal health loop iteration.

Write to `signals` table when:

- a monitored state changes
- a threshold is crossed
- a subsystem recovers to normal
- a periodic snapshot is due
- config threshold values are loaded/updated

That keeps the table useful without flooding it.

6. Metadata expected per health signal

Signal Type	Suggested Metadata
`device_heartbeat`	`heartbeat_seq`, `runtime_uptime_sec`
`device_connectivity_status`	`network_type`, `reason_code`
`battery_level`	`battery_id`, `power_source`, `threshold`
`power_supply_status`	`power_source`, `reason_code`
`camera_obstruction_status`	`camera_id`, `reason_code`, `frame_id`
`frame_brightness_contrast_quality`	`camera_id`, `frame_id`, `window_id`
`sensor_health_status`	`sensor_id`, `reason_code`
`sensor_communication_status`	`sensor_id`, `bus_id`, `reason_code`
`communication_timeout_threshold`	`device_id`, `config_version`
`battery_low_threshold`	`device_id`, `config_version`

These fields are consistent with the metadata-oriented design already present in your signal catalogue.

7. Developer implementation flow

```
Subscribe to health-relevant topics
→ Maintain latest local subsystem state
→ Run threshold / timeout / state-change checks
→ Build signal-shaped record
→ Write directly to signals table
```

8. Recommended internal structure in the node

Internal Part	Responsibility
Topic Subscribers	Consume `/system/...`, relevant `/status/...`,

Internal Part	Responsibility
	config topics
State Cache	Keep latest known subsystem state
Health Rule Evaluator	Threshold, timeout, freshness, and state-change logic
Emission Controller	Decide periodic vs change-based write
Signal Record Builder	Convert result into `signals` row format
Storage Writer	Insert into `signals` table

9. Practical examples

Example: battery level

- Sample battery input arrives
- Compare to last emitted value
- If change >= threshold or periodic interval elapsed, write `battery_level`

Example: camera obstruction

- Camera health checker flips from false → true
- Immediately write `camera_obstruction_status=true`
- On recovery, write `camera_obstruction_status=false`

Example: sensor communication

- No fresh updates within timeout
- Write `sensor_communication_status="warning"` or `"critical"`
- On recovery, write `"healthy"`